Characterizing Shadow AI Data Exfiltration Patterns in Enterprise Traffic

Rutaban Jania[1], Joya Biswas[1], Farhan Sadik Ahmed[2] and MD Fahim Hossain[3]

[1] Institute of Information Technology, Jahangirnagar University, Savar, Dhaka 1342, Bangladesh

[2] Lassonde School of Engineering, York University, 4700 Keele Street, North York, Ontario M3J 1P3, Canada

[3] Department of Anthropology, Jahangirnagar University, Savar, Dhaka 1342, Bangladesh

ayushitahrim4949@gmail.com, joyabiswas0103@gmail.com, farhansadik2002@gmail.com, hossainfahim520@gmail.com

Abstract. Enterprises now lose proprietary information through a mundane channel. Private source code and internal business logs leave over unsanctioned assistants and autonomous agents, in the same TLS sessions to the same providers as ordinary traffic. Content matching data loss prevention cannot see this traffic, because proprietary code and structured exports carry no fixed identifier, and deep inspection is not possible under TLS 1.3. This paper characterizes behaviour instead of content. A passive tap extracts TLS record headers from sessions carrying real repository source, real complaint records and real instruction prompts, and a flow representation is built from that information alone. Over 2180 sessions the representation separates benign assistant use from exfiltration with a macro F1 of 0.982 and a ROC AUC of 0.999 for the best model at a 1.33 percent false positive rate. A plaintext DLP baseline with regular expressions reaches a recall of 0.024 on the same corpus. The representation also withstands payload obfuscation and timing evasion and transfers to withheld exfiltration behaviours, reinforcing the view that structure and rhythm are harder to hide than content.

Keywords: Shadow AI, Data exfiltration, Encrypted traffic analysis, Data loss prevention, Insider threat, Network behaviour analysis.

1 Introduction

Generative assistants arrived before the controls designed to regulate them, and the fastest way to get a useful answer is to give the model the material the question is about, often a private repository, a customer export or an internal planning document. In the IBM Cost of a Data Breach Report for 2025, one in five surveyed organizations reported a breach linked to unsanctioned artificial intelligence tools, at 670 thousand dollars above the study average, and 97 percent of the breached organizations lacked proper access controls [1]. Shadow AI is that phenomenon, and it takes its shape from shadow information technology, where staff use unapproved services because approved ones are slow [2, 3].

The channel has no imperfections. The destination is a familiar business endpoint, the protocol is the same TLS 1.3 used throughout the working day [21], and the request is an ordinary chat completion, structurally identical to a developer asking what a regular expression will match. Commodity data loss prevention searches for

identifiers with fixed shapes, classification banners and credential material [5], and a private repository contains none of these. What separates a leak from a question is how much material flows, what form it takes and when it flows.

If the payload is unreadable, and would not be decisive even if it were readable, the defender should consider what encryption does not conceal. The size of each TLS record, its direction and the moment it crossed the boundary stay visible to a passive observer. Pushing a repository inverts the shape of the session, since the history is resent each turn while the answers stay short, and an automated collector adds a rhythm no person types. These are properties of the activity rather than of the text, and an insider who rewrites or encodes the text does not remove them.

No public traffic of this kind is available, so we built a measurement testbed. A local endpoint reproduces the request formats of the major commercial chat interfaces and answers with a token by token event stream; a passive tap reads the 5 byte TLS record header from the ciphertext, timestamps each record and forwards it through a shaped wide area link. The material carried is real: leaked code from two widely used open source Python repositories, leaked documents from a public corpus of consumer complaint records, benign prompts from a public instruction corpus, and enterprise traffic from UNSW-NB15 [17, 18]. Reported numbers are measurements of sessions that ran, not samples from a distribution we chose.

The paper has three contributions. The first is a repeatable measurement harness and TLS level observations, built entirely from publicly available content and released with the code, covering six assistant bound behaviours, three benign and three adversarial. The second is a traffic representation of encrypted assistant bound sessions using only record sizes, directions and arrival times, with evidence that it separates exfiltration from normal assistant use more clearly than a plaintext content matching baseline. The third is an assessment of detector properties: transfer to withheld exfiltration, robustness to payload obfuscation and timing evasion, and single flow inference cost.

2 Background and Related Work

2.1 Content Matching Data Loss Prevention

Commercial data loss prevention targets what regulation identifies. Alneyadi et al. review the techniques and the assumption behind them, that leaked data can be recognized from the shape of the data itself [5]. That assumption underpins regular expressions, checksum validation, exact matching against a registered corpus and document fingerprinting, and it holds when the table carries customer identifiers. It fails when the asset is source code whose value is what it does rather than any string it contains, and it fails again when the leak is a paraphrase, a summary or an encoding, all of which are perfectly legitimate things to ask an assistant for.

2.2 Encrypted Traffic Analysis

The study of what remains visible after encryption is well developed. Papadogiannaki and Ioannidis survey it and note its central paradox, that the metadata helping a defender identify malware also helps an adversary profile a user [6]. Website

fingerprinting shows how much packet size and direction recover at internet scale and against defended traffic [8, 9, 10]. Anderson and McGrew classify encrypted malware command channels from flow features under noisy labels and shifting distributions [7], convolutional and recurrent architectures have been applied to application classification on packet sequences [11], and time related features separate tunneled from direct traffic [12].

Two recent studies show how visible assistant traffic is from outside. Whisper Leak uses the size and timing of packets in the streamed response to infer the topic of a conversation with a remote language model across 28 models and several providers [13], and NetEcho recovers much of a conversation from the same side channel [14]. Both treat the leakage as a privacy problem to be solved. The present work inverts the frame. If token by token streaming makes a session legible to a passive observer, the defender at the enterprise egress can use that structure to tell a question from a leak. To the best of our knowledge this direction has not been tested.

2.3 Security of Large Language Model Deployments

Work on deployment security has focused on the model and the application around it. Extraction attacks showed that a deployed model can emit memorized training data [16], and retrieved content can steer an assistant into sending user data to an attacker controlled endpoint [15], the second most common attack type in the 2025 OWASP list [4]. All of this concerns what the model does once it receives data. Less attention has gone to how the data reached the model, and whether the enterprise could have noticed it leaving.

3 Threat Model

The adversary is an insider and is not necessarily malicious. The canonical case is an engineer pasting a private service into an open assistant file by file, or an analyst exporting a customer table and asking for a summary. An unsanctioned agent runtime with a personal key and a hostile insider are covered by the same model, because all of them lead to one observable: a large amount of material moving outward in assistant requests. The adversary is not attacking the model or the network. Both are used as intended.

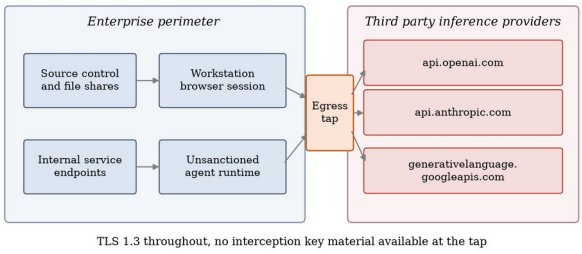


Fig. 1. Threat model. Internal repositories and service endpoints feed a browser session or an unsanctioned agent runtime, both of which reach third party providers over TLS 1.3. The defender observes the egress path and holds no key material.

The defender is a passive tap on the egress path. It reads every byte of ciphertext in both directions with accurate timestamps, along with the destination address and the server name, and it cannot decrypt. This setting is deliberately the hard one. Organizations that terminate TLS can also inspect content, and Section 5.5 shows what that buys them. We hold that metadata is sufficient, so that the method also applies where interception is impossible for legal or technical reasons.

We assume the defender sees traffic bound for inference endpoints, since providers publish their server names and those names are visible at the tap. The question that follows is when assistant sessions are questions and when they are leaks. We do not assume the adversary pads traffic to a fixed transport layer profile: the adversary may encode, compress or reorder the payload, but doing so at the transport layer is beyond a browser session.

4 Methodology

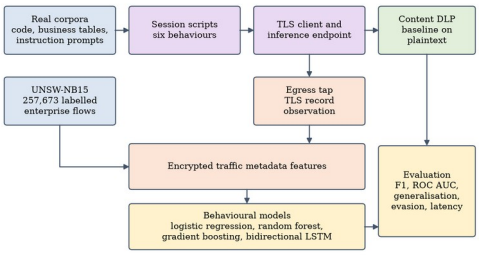


Fig. 2. Measurement and evaluation pipeline. Plaintext reaches only the content matching baseline, never the behavioural models.

4.1 Testbed and Observation Point

The request body matches the OpenAI chat completion and Anthropic messages interfaces, and the downlink is an event stream carrying one event per generated token, as in a streaming response. Response length is capped sublinearly in request length, so the assistant summarizes rather than restates, the token rate runs between 38 and 92 tokens per second per request, and the text on the wire is real language from a public instruction corpus rather than filler.

A client opens one TLS connection per session and reuses it, as browsers do, carrying ordinary HTTP requests with a JSON body. The tap sits between the endpoints and never decrypts. It records the ciphertext, reads the 5 byte TLS record header, stores each boundary position, record direction and record length, and forwards the record after a delay set by the link model. That model serializes each direction independently at 40 megabits per second up and 200 megabits per second down and adds a one way propagation delay drawn from a normal distribution with mean 14 ms and standard deviation 3.5 ms. Serialization is additive under pipelining while propagation is not, as on a real path. Where packet counts are needed, records map to segments at a maximum segment size of 1448 bytes.

The sizes are therefore what the TLS implementation emitted for the payloads sent, and the times are those of sessions that ran. The only model supplied elements are the shaping of the emulated link and the token cadence of the endpoint.

4.2 Session Behaviors and Content

Table 1 lists the six scripted behaviours, three benign and three adversarial. Benign chat is a one to eight turn conversation from the Alpaca corpus [28] with pauses drawn from a lognormal reading and typing distribution. Benign document question answering pastes an excerpt of one to five kilobytes and asks about it, the common case a naive volume threshold would flag. Chunked code leak breaks real repository files into pieces of two to sixteen kilobytes separated by short intervals of machine activity. Bulk document leak sends one to three very large complaint record documents in a single request. Agentic scraping sends twelve to sixty requests carrying a table slice and a host inventory at a nearly constant interval. Obfuscated code leak repeats the chunked leak with the payload encoded as base sixty four, compressed base sixty four, hexadecimal or reversed lines, with timing jitter or randomized chunk sizes.

Table 1. Session behaviours, content source and session count.

Behaviour	Label	Content source	Sessions
Benign chat	Benign	Alpaca instruction corpus	620
Benign document question answering	Benign	Repository files and complaint records	380
Chunked code leak	Exfiltration	Flask and Requests source	340
Bulk document leak	Exfiltration	Consumer complaint records	300
Agentic scraping	Exfiltration	Repository inventory and table slices	300
Obfuscated code leak	Exfiltration	Flask and Requests source, encoded	240

4.3 Feature Construction

The features come only from the record tuples and fall into four groups. Volume and asymmetry features capture how much moves in each direction, such as the uplink share of session bytes, and the difference between the logarithms of the two volumes. Record structure features describe the size distribution through the mean, the ninetieth percentile, the coefficient of variation, a coarse entropy over quantized sizes and the fraction of uplink records at maximum size. Temporal features cover interarrival statistics and jitter. Burst features split the uplink into segments and report the burst count, the largest burst volume, the mean span, the variation of uplink gaps between bursts and the longest run of uplink records with no downlink record between them. The full vector has length 51. A parallel sequence view is kept for the recurrent model, giving each record a signed logarithmic size, the logarithm of the gap to the previous record and a direction indicator.

4.4 Background Enterprise Traffic

Assistant bound sessions alone cannot say whether a monitor would drown in alerts on a normal network, so we pair the measured flows with UNSW-NB15, which supplies labelled enterprise sessions for normal activity and nine attack families [17, 18]. Both public partitions are used, giving 257,673 records. After converting the UNSW interarrival and jitter columns to seconds and recomputing the asymmetry terms as in the first source, 18 features are common to both. The space is built carefully, since a feature present in one source and absent in the other would let a classifier learn the generator rather than the behaviour.

4.5 Models and Baseline

The linear reference is logistic regression on standardized features, the nonlinear reference is a random forest of four hundred trees with balanced subsample weighting [22], and the boosted ensemble is histogram based gradient boosted trees [23]. For the sequence view, a bidirectional long short term memory network with sixty four hidden units, max pooling over time and a two layer head is trained for sixty epochs with class weighted cross entropy [24, 25]. The implementations come from scikit-learn and PyTorch [26, 27]. The baseline is a content matching engine of the kind most commodity data loss prevention products provide, with nine rules covering payment card numbers, national identifiers, cloud access keys, private key blocks, provider tokens, bank account numbers, classification banners, mail addresses and telephone numbers. Seven of them raise an alert, while addresses and telephone numbers are treated as telemetry, since an alert on those is not useful inside a corporate network. The baseline is run against the plaintext of every session, which assumes the full TLS interception the behavioural models are denied.

5 Evaluation

5.1 Protocol

All sessions were produced on a single host with Python 3.14.4, scikit-learn 1.9.0 and PyTorch 2.14.0. Splits are stratified with the seed fixed at 20260904. Section 5.4 reports fivefold cross validation and a seventy to thirty holdout, reused by every later experiment so that all comparisons rest on the same sessions. We report macro averaged F1 with precision and recall, plus the areas under the receiver operating characteristic and precision recall curves, since a monitor judged on accuracy alone looks excellent and misses what matters.

5.2 How the Behaviors Separate

Before fitting any classifier it is worth asking whether there is any separation at all. Table 2 gives medians by behaviour and Fig. 3 shows the sessions themselves.

Table 2. Median per behaviour of selected measured quantities. Volumes are in bytes and durations in seconds.

Behaviour	Uplink bytes	Downlink bytes	Uplink share	Seconds	Uplink records	Longest run
Benign	5,412	69,740	0.070	52.7	11	3

Behaviour	Uplink bytes	Downlink bytes	Uplink share	Seconds	Uplink records	Longest run
chat						
Benign document question answering	9,828	68,872	0.133	27.3	9	3
Chunked code leak	107,602	201,923	0.345	33.5	16	4
Bulk document leak	558,354	188,139	0.760	26.9	40	25
Agentic scraping	301,418	650,740	0.317	194.3	77	3
Obfuscated code leak	83,818	192,031	0.317	38.2	15	3

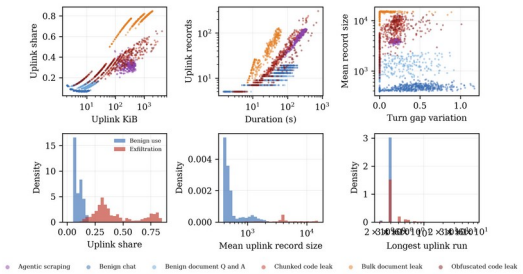


Fig. 3. Top, every measured session in three projections. Bottom, marginal distributions of three features for benign use and for exfiltration, with zero valued sessions excluded.

The pattern is the one the threat model predicts. Benign conversation is download heavy, since a short question earns a long streamed answer, while the leak behaviours invert the ratio by more than the raw volume difference, because the history is resent every turn. Agentic scraping sits between the two on volume and is distinguished by pacing and burst count. Fig. 4 shows raw record traces, where the difference in texture is visible without any statistics.

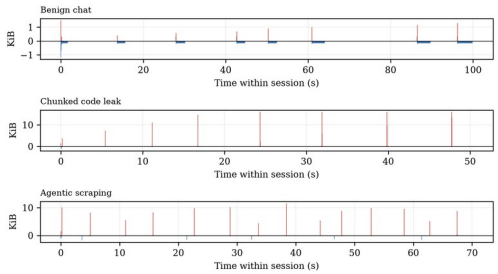


Fig. 4. TLS record traces of three sessions. Bars above the axis are uplink records and bars below are downlink records.

5.3 Three Class Detection in the Harmonized Space

The first experiment asks whether shadow AI exfiltration can be separated from ordinary enterprise traffic and from attack traffic using only the eighteen harmonized features. The pool holds 11,000 benign records, 10,000 legacy threat records and 1,180 exfiltration sessions, split seventy to thirty, with results in Table 3 and the confusion structure in the left pane of Fig. 6.

Table 3. Three class results on the harmonised feature space. The exfiltration columns are that class against the rest.

Model	Accuracy	Macro F1	Exfiltration recall	Exfiltration precision	ROC AUC
Logistic Regression	0.786	0.826	0.997	0.863	1.000
Random Forest	0.916	0.936	0.975	0.994	1.000
Gradient Boosting	0.922	0.939	0.989	0.975	1.000

The best model reaches a macro F1 of 0.939 and a recall of 0.989 on the exfiltration class. The result must be read carefully, since the three classes come from two generators and residual domain differences cannot be ruled out, even though the harmonized space holds only features defined identically by both. We therefore treat it as evidence that assistant bound exfiltration resembles neither ordinary traffic nor the UNSW-NB15 attack families, and put the weight on the next experiment, where one generator produces every session.

5.4 Benign Assistant Use Against Exfiltration

This is the operationally relevant task. Every session targets an inference endpoint and came from the same harness and link, so the classifier has to decide which of them are leaks. The set holds 1000 benign and 1180 exfiltration sessions, reported in Table 4.

Table 4. Assistant bound binary task. The first two columns are five fold cross validation with the standard deviation across folds in parentheses, the rest are the holdout split.

Model	CV macro F1	CV ROC AUC	Recall	Precision	Macro F1	ROC AUC	PR AUC
Logistic Regression	0.979 (0.003)	0.999	0.972	0.986	0.977	0.998	0.999
Random Forest	0.982 (0.004)	0.999	0.977	0.989	0.982	0.999	0.999
Gradient Boosting	0.978 (0.004)	0.999	0.972	0.983	0.975	0.998	0.999
Bidirectional LSTM	not run	not run	0.901	0.894	0.888	0.975	0.980

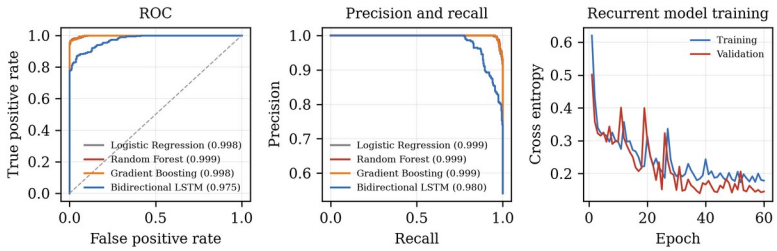


Fig. 5. Holdout ROC and precision recall curves with areas in the legends, and the loss curves of the recurrent model.

The random forest is the best model, with a macro F1 of 0.982 and a ROC AUC of 0.999. A single feature separates the bulk leak cleanly, while agentic scraping and short chunked leaks need the combinations of pacing and structure the tree ensembles supply. The recurrent model, which uses no aggregate feature and sees only the first 128 record sizes and gaps, reaches a macro F1 of 0.888 and a ROC AUC of 0.975. It trails the tabular models, since a truncated sequence does not carry what the aggregates carry, yet it is strong evidence that most discriminative information lies in the shape of the session rather than in any hand selected summary. Its training curves appear on the right of Fig. 5.

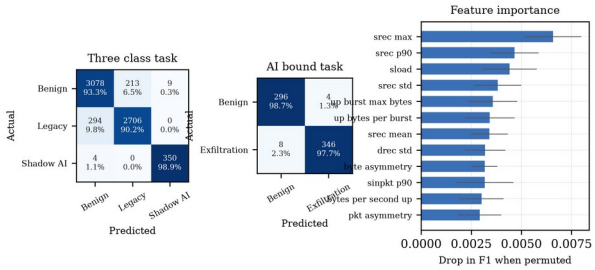


Fig. 6. Confusion matrices for the three class and assistant bound tasks, with counts and row normalised percentages, and the twelve strongest features by permutation importance.

The right side of Fig. 6 ranks features by the drop in F1 when the feature is permuted, and the five strongest are srec max, srec p90, sload, srec std and up burst max bytes. These are the quantities the threat model said should matter, which is a good sign that the classifier is not keying on an artefact of the harness.

5.5 Comparison Against Content Matching

The content matching baseline was given the plaintext of every session, more than any deployment without full interception receives. On the holdout split it reaches a recall of 0.017 at a precision of 1.000, while the left panel of Fig. 7 shows the random forest reaching a precision of 0.989 at a recall of 0.977 from metadata alone. Only two rules fired, a classification banner in 30 sessions and an electronic mail address in 68. The gap is structural rather than a matter of tuning. The leaked material carries no payment card number, national identifier or private key block, none of which

normally appear in a working web framework or a table of complaint narratives, and the rules that do fire are generic and fire at a similar rate on benign sessions, so raising their severity would trade recall for precision. This is not a weakness of the implementation: it follows from the fact that the value of proprietary material is not embedded in any string it contains.

5.6 Transfer to Unseen Exfiltration Behaviour

A detector trained only on the behaviours it was shown will miss the next insider who does something slightly different. We remove one exfiltration behaviour from training, fit the random forest on the rest, and test the held out behaviour against reserved benign sessions, with results in the right panel of Fig. 7.

The strongest transfer is bulk document leak at 1.000 and the weakest is chunked code leak at 0.968. The ordering is explicable. Behaviours whose coarse property is sustained outward volume are inferred more readily from volume driven examples than behaviours distinguished by pacing. A deployment therefore needs one example of each broad family rather than one of every variant, which is a much weaker requirement.

5.7 Robustness to Obfuscation and Timing Evasion

The answer to content inspection is that an insider can encode the payload, and the answer to behavioural detection is that an insider can slow down. We train only on the non evasive sessions, keep a quarter as a reference and test on three families of evasive sessions. Payload obfuscation encodes the leaked code as base sixty four, compressed base sixty four, hexadecimal or reversed lines, timing jitter replaces machine pacing with the lognormal think time distribution of the benign behaviours, and chunk morphing randomizes the size of each uploaded piece. The outcome is in the centre panel of Fig. 7.

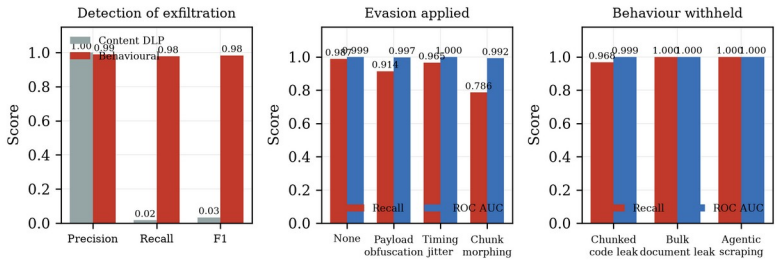


Fig. 7. Left, the content matching baseline against the behavioural model on the same holdout sessions. Centre, robustness under three evasion families. Right, transfer to an exfiltration behaviour withheld from training.

The model, which never saw the evasive families, scores 0.987 on non evasive sessions, against 0.914 for payload obfuscation, 0.965 for timing jitter and 0.786 for chunk morphing. Obfuscation sharpens the upload asymmetry the model keys on rather than hiding it, since base sixty four makes the payload a third larger and hexadecimal doubles it, at no cost to the insider. The strongest of the three is chunk morphing, which destroys the record size statistics ranked highest in Fig. 6 and cuts

recall to 0.786. Even there the area under the curve is 0.992, so the sessions stay separable at a lower decision threshold, and the false positive rate on the reserved benign sessions is 0.0 percent, which leaves room to lower it. The reading is that evasion makes the defender's threshold harder to set faster than it makes the representation easier to escape.

The mean time to score one completed flow with the random forest is 24.82 milliseconds, with a 95th percentile of 26.21 and a 99th percentile of 26.62. Feature extraction is a single pass over the record list, so the classifier is not the slow piece. Since the decision waits for the end of the session, this is a near real time analytic on completed flows rather than a block.

6 Discussion and Limitations

The results support a limited hypothesis. Passively observed sizes, directions and arrival times of the TLS records in an assistant session distinguish a person asking a question from a person or process moving internal material out, across three exfiltration behaviours and four payload encodings, with no access to plaintext.

The strongest limitation is the traffic source. It is measured rather than sampled, a large improvement over a synthetic table, but it runs against a local endpoint over an emulated link rather than a commercial provider over the public internet, the token cadence is our stand in for a provider, and the link sits at one operating point. Recent measurement work confirms that real streaming responses produce the per token record structure our endpoint reproduces [13, 14], which has not yet been studied against live providers, so the absolute numbers here are not quoted as operational.

The second limitation is the benign class. Real networks carry heavier legitimate traffic, such as an approved coding assistant indexing a repository under contract, which would closely resemble our exfiltration class. Flow shape is not a policy separation; a deployment would separate the two on policy, using the destination and the credential. The detector responds to the occurrence of a bulk transfer, not to its authorization.

Third, a determined opponent has options we did not model. Rounding to a fixed size would break the representation, and padding is discussed at length in the fingerprinting literature [8, 9, 10]. The cheapest evasions are tested here and the method survives them, but an adversary willing to install tooling can do better, while a casual leak becomes far more costly. Fourth, the method decides after the session ends, so it detects rather than prevents. The recurrent model is a partial remedy through incremental scoring on a sliding window, which we leave to future work.

Finally, a defensive technique built on traffic metadata is also a surveillance technique, since it reveals how often an employee uses an assistant and what leaves the repository [6, 13, 14]. An organization deploying it should restrict the analytic to aggregate flow records, keep sequences only as long as the decision requires, and make the practice public.

7 Conclusion and Future Work

Proprietary content leaves organizations through a path that looks entirely normal. This paper argued that the problem should be reframed away from reading the payload and towards characterizing the session. We built an instrumented testbed in which real records from real business sources move to a real inference endpoint over TLS, ran 2180 sessions at record level, and showed that features built from record size, direction and arrival time separate exfiltration from benign assistant use with a macro F1 of 0.982, where a content matching baseline granted full plaintext reached a recall of 0.024. The representation transferred to withheld exfiltration behaviours and withstood payload obfuscation.

Three directions follow: measurement against live commercial providers over varied network paths, incremental scoring so that a decision can be taken part way through a long transfer, and the opposing question of which padding and pacing strategies defeat this representation and what they cost the insider. The testbed, the measured corpus and the analysis code are released so that each can be pursued from the same baseline.

Acknowledgements and availability.

The authors thank the maintainers of UNSW-NB15 and of the public corpora used to build the payloads. The harness, the derived flow corpus and all analysis code are available at https://github.com/rutabanjania/shadow-ai-exfiltration.

References

1. IBM Security: Cost of a Data Breach Report 2025. IBM Corporation (2025). https://www.ibm.com/reports/data-breach
2. Silic, M., Silic, D., Kind-Truller, K.: From shadow IT to shadow AI. Threats, risks and opportunities for organizations. Strategic Change (2025). https://doi.org/10.1002/jsc.2682
3. Silic, M., Back, A.: Shadow IT. A view from behind the curtain. Computers and Security 45, 274-283 (2014). https://doi.org/10.1016/j.cose.2014.06.007
4. OWASP Foundation: OWASP Top 10 for Large Language Model Applications, version 2025. OWASP (2025). https://owasp.org/www-project-top-10-for-large-language-model-applications/
5. Alneyadi, S., Sithirasenan, E., Muthukkumarasamy, V.: A survey on data leakage prevention systems. Journal of Network and Computer Applications 62, 137-152 (2016). https://doi.org/10.1016/j.jnca.2016.01.008
6. Papadogiannaki, E., Ioannidis, S.: A survey on encrypted network traffic analysis applications, techniques, and countermeasures. ACM Computing Surveys 54(6), 1-35 (2022). https://doi.org/10.1145/3457904
7. Anderson, B., McGrew, D.: Machine learning for encrypted malware traffic classification. Accounting for noisy labels and non stationarity. In: Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 1723-1732. ACM (2017). https://doi.org/10.1145/3097983.3098163
8. Sirinam, P., Imani, M., Juarez, M., Wright, M.: Deep fingerprinting. Undermining website fingerprinting defenses with deep learning. In: Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security, pp. 1928-1943. ACM (2018). https://doi.org/10.1145/3243734.3243768

9. Panchenko, A., Lanze, F., Zinnen, A., Henze, M., Pennekamp, J., Wehrle, K., Engel, T.: Website fingerprinting at internet scale. In: Proceedings 2016 Network and Distributed System Security Symposium. Internet Society (2016). https://doi.org/10.14722/ndss.2016.23477
10. Rimmer, V., Preuveneers, D., Juarez, M., Van Goethem, T., Joosen, W.: Automated website fingerprinting through deep learning. In: Proceedings 2018 Network and Distributed System Security Symposium. Internet Society (2018). https://doi.org/10.14722/ndss.2018.23105
11. Lotfollahi, M., Jafari Siavoshani, M., Shirali Hossein Zade, R., Saberian, M.: Deep packet. A novel approach for encrypted traffic classification using deep learning. Soft Computing 24(3), 1999-2012 (2020). https://doi.org/10.1007/s00500-019-04030-2
12. Draper-Gil, G., Lashkari, A.H., Mamun, M.S.I., Ghorbani, A.A.: Characterization of encrypted and VPN traffic using time related features. In: Proceedings of the 2nd International Conference on Information Systems Security and Privacy, pp. 407-414. SCITEPRESS (2016). https://doi.org/10.5220/0005740704070414
13. McDonald, G., Bar Or, J.: Whisper leak. A side channel attack on large language models. arXiv preprint arXiv:2511.03675 (2025). https://arxiv.org/abs/2511.03675
14. Zhang, Z., Wu, G., Deng, S., Wang, S., Zhang, Y.: NetEcho. From real world streaming side channels to full LLM conversation recovery. arXiv preprint arXiv:2510.25472 (2025). https://arxiv.org/abs/2510.25472
15. Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., Fritz, M.: Not what you have signed up for. Compromising real world LLM integrated applications with indirect prompt injection. In: Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security, pp. 79-90. ACM (2023). https://doi.org/10.1145/3605764.3623985
16. Nasr, M., Carlini, N., Hayase, J., Jagielski, M., Cooper, A.F., Ippolito, D., Choquette-Choo, C.A., Wallace, E., Tramer, F.: Scalable extraction of training data from production language models. arXiv preprint arXiv:2311.17035 (2023). https://arxiv.org/abs/2311.17035
17. Moustafa, N., Slay, J.: UNSW-NB15. A comprehensive data set for network intrusion detection systems. In: 2015 Military Communications and Information Systems Conference, pp. 1-6. IEEE (2015). https://doi.org/10.1109/MilCIS.2015.7348942
18. Moustafa, N., Slay, J.: The evaluation of network anomaly detection systems. Statistical analysis of the UNSW-NB15 data set and the comparison with the KDD99 data set. Information Security Journal. A Global Perspective 25(1-3), 18-31 (2016). https://doi.org/10.1080/19393555.2015.1125974
19. Sharafaldin, I., Habibi Lashkari, A., Ghorbani, A.A.: Toward generating a new intrusion detection dataset and intrusion traffic characterization. In: Proceedings of the 4th International Conference on Information Systems Security and Privacy, pp. 108-116. SCITEPRESS (2018). https://doi.org/10.5220/0006639801080116
20. Paxson, V.: Bro. A system for detecting network intruders in real time. Computer Networks 31(23-24), 2435-2463 (1999). https://doi.org/10.1016/S1389-1286(99)00112-7
21. Rescorla, E.: The Transport Layer Security (TLS) protocol version 1.3. RFC 8446, Internet Engineering Task Force (2018). https://doi.org/10.17487/RFC8446
22. Breiman, L.: Random forests. Machine Learning 45(1), 5-32 (2001). https://doi.org/10.1023/A:1010933404324
23. Chen, T., Guestrin, C.: XGBoost. A scalable tree boosting system. In: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 785-794. ACM (2016). https://doi.org/10.1145/2939672.2939785

24. Hochreiter, S., Schmidhuber, J.: Long short term memory. Neural Computation 9(8), 1735-1780 (1997). https://doi.org/10.1162/neco.1997.9.8.1735
25. Graves, A., Schmidhuber, J.: Framewise phoneme classification with bidirectional LSTM and other neural network architectures. Neural Networks 18(5-6), 602-610 (2005). https://doi.org/10.1016/j.neunet.2005.06.042
26. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., Duchesnay, E.: Scikit-learn. Machine learning in Python. Journal of Machine Learning Research 12, 2825-2830 (2011)
27. Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., Chintala, S.: PyTorch. An imperative style, high performance deep learning library. In: Advances in Neural Information Processing Systems 32, pp. 8024-8035. Curran Associates (2019)
28. Taori, R., Gulrajani, I., Zhang, T., Dubois, Y., Li, X., Guestrin, C., Liang, P., Hashimoto, T.B.: Stanford Alpaca. An instruction following LLaMA model. GitHub repository (2023). https://github.com/tatsu-lab/stanford_alpaca